

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

SEMINARSKI RAD
iz predmeta
Bioinformatika 1

Bamboo Filter

Implementacija filtera dinamičke veličine za pretraživanje k-mera u genomu

Autori:

Antonio Šimić

Jakov Malić

Akadska godina: 2025./2026.

Zadatak: (2) Bamboo Filter [1, 2]

Zagreb, svibanj 2026.

Sažetak

U ovom radu prikazana je implementacija *Bamboo* filtera, vjerojatnosne strukture podataka za odgovaranje na upite o pripadnosti skupu (engl. *Approximate Membership Query*, AMQ), opisane u radovima Wang *et al.* [1, 2]. Glavna prednost Bamboo filtera u odnosu na klasični Cuckoo Filter [3] jest mogućnost glatkog (engl. *smooth*) povećavanja i smanjivanja kapaciteta bez potrebe za potpunom rekonstrukcijom strukture. Filter je implementiran u programskom jeziku C++17, prema Google C++ stilskim smjernicama, te testiran na umjetno generiranim DNK nizovima duljine 10^3 – 10^6 baznih parova i na cjelovitom genomu bakterije *Escherichia coli* K-12 MG1655 (4,64 Mbp). Za svaku duljinu i veličinu k -mera ($k \in \{10, 20, 50, 100, 200\}$) izmjereni su vrijeme umetanja, vrijeme pretraživanja, lažno-pozitivna stopa (FPR) te zauzeće memorije. Rezultati su uspoređeni s referentnom implementacijom autora [10]: naša implementacija postiže FPR od 0,08–0,46% (oko 2 – $4\times$ više od reference), memorijski je u paritetu (32–41 bit po elementu, $\pm 10\%$ reference) te je vremenski *brža* u većini konfiguracija (40–60% referentnog vremena umetanja za $k \geq 50$).

Ključne riječi: Bamboo Filter, Cuckoo Filter, AMQ struktura, k -meri, bioinformatika, dinamičko dimenzioniranje, hash adresiranje.

Sadržaj

1	Uvod	1
2	Teorijska pozadina	1
2.1	Bloomov filter	1
2.2	Cuckoo Filter	2
2.3	Potreba za dinamičkim dimenzioniranjem	2
3	Bamboo Filter	2
3.1	Glavne ideje	2
3.2	Struktura podataka	3
3.3	Dekompozicija hash vrijednosti	3
3.4	Operacija pretraživanja (Lookup)	4
3.5	Operacija umetanja (Insert)	4
3.6	Operacija brisanja (Delete)	5
3.7	Glatko dijeljenje (Smooth Expansion)	5
3.8	Glatko smanjivanje (Smooth Shrinking)	6
3.9	Primjer na malim podacima	6
4	Implementacija	6
4.1	Struktura projekta	6
4.2	Tehnologije i ovisnosti	7
4.3	Doprinosi članova tima	7
4.4	Upute za izvođenje	8
5	Eksperimentalni rezultati	8
5.1	Metodologija	8
5.2	Testno okruženje	9
5.3	Rezultati na sintetičkim podacima	9
5.4	Rezultati na genomu E. coli	10
5.5	Analiza točnosti (FPR)	11
5.6	Analiza memorije	12
5.7	Validacija ispravnosti	13
5.8	Usporedba s referentnom implementacijom	13
6	Zaključak	15

1 Uvod

U mnogim primjenama bioinformatike potrebno je brzo odgovarati na pitanje pripada li neki kratki podniz (k -mer) skupu podniza prethodno indeksiranih iz nekog genoma. Tipičan slučaj uporabe javlja se kod usporedbe sekvenci, detekcije varijanti, sastavljanja genoma i klasifikacije metagenomskih očitavanja. Eksplicitno pohranjivanje svih k -mera u hash tablici je memorijski preskupo — za genom *E. coli* duljine $4,64 \cdot 10^6$ baznih parova broj različitih 20-mera iznosi reda veličine milijuna, a za eukariotske genome (npr. ljudski genom od $3 \cdot 10^9$ bp) broj se penje na milijarde.

Klasično rješenje ovog problema su vjerojatnosne AMQ strukture (*Approximate Membership Query*), koje za cijenu male lažno-pozitivne stope (engl. *false positive rate*, FPR) drastično smanjuju zauzeće memorije. Najpoznatiji predstavnici su Bloomov filter [5], Cuckoo Filter [3] i njihove varijante. Zajednički im je nedostatak *statička veličina*: nakon inicijalizacije, povećanje kapaciteta zahtijeva alokaciju nove, veće strukture i ponovno umetanje svih elemenata, što je u praksi neprihvatljivo za tijekove podataka nepoznate veličine (engl. *streaming*).

Bamboo Filter (BF), kojeg su 2022. godine predstavili Wang *et al.* [1] (s proširenjima u [2]), rješava upravo taj problem: omogućuje *glatko* (postupno, po jedan segment u jednoj operaciji) povećavanje kapaciteta uz amortizirano konstantnu cijenu po umetanju, te analogno smanjivanje pri brisanju elemenata. U ovom radu prikazujemo vlastitu implementaciju Bamboo filtera u jeziku C++ te ju eksperimentalno procjenjujemo na umjetno generiranim DNK nizovima i na referentnom genomu bakterije *E. coli* K-12 MG1655 [9].

Cilj rada je:

1. implementirati Bamboo filter prema specifikaciji iz [1],
2. demonstrirati ispravnost implementacije kroz jedinične testove,
3. analizirati performanse (vrijeme, memorija, FPR) na sintetičkim i stvarnim biološkim podacima,
4. usporediti dobivene rezultate s referentnom implementacijom autora [10].

2 Teorijska pozadina

2.1 Bloomov filter

Bloomov filter [5] pohranjuje članstvo elemenata u skupu S pomoću bit-niza duljine m i h neovisnih hash funkcija $h_1, \dots, h_h$. Pri umetanju elementa x postavljaju se na 1 bitovi na pozicijama $h_i(x) \bmod m$ za $i = 1, \dots, h$. Pri provjeri članstva element se smatra prisutnim

ako su svi pripadajući bitovi postavljeni na 1. Bloomov filter ne podržava brisanje, ima fiksnu veličinu i FPR raste s popunjenošću.

2.2 Cuckoo Filter

Cuckoo Filter [3] predstavlja unaprijeđenje Bloomovog filtera koje podržava brisanje uz nižu FPR pri istom utrošku memorije. Umjesto bitova, pohranjuje kratke *otiske* (engl. *fingerprints*) elementa u tablici podijeljenoj na *bucket*-e fiksnog kapaciteta (tipično 4 utora). Svaki element ima dva kandidatska bucket-a izračunata kao:

$$b_1 = \text{hash}(x) \bmod n, \quad (1)$$

$$b_2 = b_1 \oplus \text{hash}(\text{tag}) \bmod n, \quad (2)$$

gdje je n broj bucket-a, a tag otisak elementa. Pri umetanju koristi se *cuckoo* mehanizam: ako su oba bucket-a puna, jedan postojeći otisak se *izbacuje* (engl. *evict*) i premješta u svoj alternativni bucket; postupak se ponavlja dok se ne pronađe slobodan utor ili se ne dosegne maksimalan broj iteracija.

Glavni nedostatak Cuckoo filtera u kontekstu ovog rada je upravo *statička* priroda: jednom inicijaliziran sa n bucket-a, kapacitet je nepromjenjiv. Povećavanje zahtijeva alokaciju nove tablice i ponovno hashiranje svih otisaka.

2.3 Potreba za dinamičkim dimenzioniranjem

U bioinformatičkim primjenama unaprijed često ne znamo broj različitih k -mera koje ćemo umetnuti. Procjena unaprijed može biti pogrešna: premali filter dovodi do prevelike FPR, a preveliki nepotrebno troši memoriju. Dinamičke varijante kao što su *Dynamic Cuckoo Filter* [6] i *Logarithmic Dynamic Cuckoo Filter* [7] rješavaju ovaj problem dodavanjem novih razina tablica, ali pritom uvode dodatne probe pri pretraživanju. Bamboo Filter pristupa problemu drugačije — postupnim dijeljenjem postojećih segmenata.

3 Bamboo Filter

3.1 Glavne ideje

Bamboo Filter [1] predstavlja proširenje Cuckoo filtera s podrškom za *glatko* dinamičko dimenzioniranje. Ključne ideje su:

1. Memorija je podijeljena na *segmente* fiksne veličine. Svaki segment je mali Cuckoo filter s vlastitim bucket-ima i utorima.

2. Pri prelasku zadanog praga popunjenosti (eksperimentalno 90%) izvodi se *dijeljenje* (engl. *split*) jednog segmenta — doda se novi segment, a otisci se redistribuiraju između originalnog i novog na temelju jednog bita otiska.
3. Pri padu popunjenosti ispod donjeg praga (eksperimentalno 40%) izvodi se *spajanje* (engl. *merge*) zadnjeg segmenta s njegovim „roditeljem”.
4. Obje operacije se izvode *lokalno*: dodiruju samo dva segmenta, čime je amortizirana cijena umetanja $O(1)$.

3.2 Struktura podataka

Filter je organiziran kao niz segmenata $S_0, S_1, \dots, S_{N-1}$, gdje je svaki segment niz od B bucket-a, a svaki bucket polje od w utora veličine f bita. Inicijalni broj segmenata N_0 mora biti potencija broja 2. U našoj implementaciji korišteni su parametri prikazani u Tablici 1.

Tablica 1: Parametri implementacije Bamboo filtera.

Parametar	Vrijednost	Opis
B (bucket-a/segment)	64	broj bucket-a u jednom segmentu
w (utora/bucket)	4	kapacitet jednog bucket-a
f (bita/otisak)	12	duljina otiska elementa
N_0 (poč. segmenata)	$4-n/1024$	ovisno o očekivanom broju elemenata
τ_e (prag prošir.)	0,90	gornji prag popunjenosti
τ_s (prag smanj.)	0,40	donji prag popunjenosti
$k_{\max}$ (cuckoo poth.)	500	maksimalan broj izbacivanja

Vrijednost 0xFFFF koristi se kao oznaka praznog utora (engl. *sentinel*), umjesto uobičajene nule, kako bi se izbjegla potreba za remapiranjem otisaka s vrijednošću 0 (što bi narušilo bit-razinu dosljednosti s hash vrijednošću potrebnom u operaciji dijeljenja).

3.3 Dekompozicija hash vrijednosti

Za svaki ključ x izračunava se 64-bitna hash vrijednost $H(x)$ pomoću funkcije MurmurHash3 [8]. Bitovi rezultata dijele se na tri segmenta prema sljedećoj shemi:

- bitovi $[0, 6)$ — indeks bucket-a unutar segmenta ($\log_2 B = 6$),
- bitovi $[6, 6 + \log_2 N_0)$ — indeks segmenta na razini 0,
- bitovi $[6 + \log_2 N_0, 6 + \log_2 N_0 + 12)$ — 12-bitni otisak.

Bitovi otiska *namjerno se preklapaju* s bitovima koji nakon dijeljenja postaju visoki bitovi indeksa segmenta. Ova podudarnost ključna je za algoritam dijeljenja (potpoglavlje 3.7): bit L otiska na razini L jednoznačno određuje pripada li otisak izvornom ili novom (engl. *buddy*) segmentu.

3.4 Operacija pretraživanja (Lookup)

Pretraživanje elementa x izvodi se u tri koraka: (1) izračun hash vrijednosti i dekompozicija u otisak t , indeks segmenta s i primarni bucket b_1 ; (2) izračun alternativnog bucket-a $b_2 = b_1 \oplus h(t)$ unutar istog segmenta; (3) linearno pretraživanje sva 4 utora u oba bucket-a. Element se smatra prisutnim ako je t pronađen u bilo kojem utoru.

3.5 Operacija umetanja (Insert)

Algoritam umetanja prikazan je u pseudokodu 1. Ključni detalji su:

- **Deduplikacija** (linije 4–7): provjerava se sadrži li bilo koji od dva kandidatska bucket-a već isti otisak. Bez ove provjere, ponavljani umeci istog k -mera (česti u stvarnim genomima zbog repetitivnih regija) doveli bi do beskonačnog dijeljenja: budući da duplikati dijele *cijeli* otisak, dijeljenje ih ne može razdvojiti, pa popunjenost samo raste.
- **Cuckoo izbacivanje** (linija 13): pri sudaru, postojeći otisak se izbacuje u svoj alternativni bucket. Postupak se ponavlja do $k_{\max} = 500$ puta. Prije početka pravimo *sigurnosnu kopiju* segmenta kako bismo u slučaju neuspjeha mogli vratiti izvorno stanje (inače bi zadnji prenošeni otisak bio trajno izgubljen).
- **Pre-emptivno dijeljenje** (linija 16): ako popunjenost pređe τ_e , odmah pozivamo `SplitSegment` kako sljedeći umetak ne bi morao prolaziti cijelu eviction petlju.

```

1 bool BambooFilter::Insert(const string& key) {
2     HashTag h = Hash64(key);
3     uint16_t tag = MakeTag(h.h1);
4     size_t s = SegmentIndex(h.h1);
5     size_t b1 = BucketIndex(h.h1);
6     size_t b2 = AltBucket(b1, tag);
7     // Deduplikacija: ako otisak već postoji, nema sto raditi.
8     if (BucketContains(s, b1, tag) || BucketContains(s, b2, tag))
9         return true;
10    while (true) {
11        if (TryInsertOnce(tag, s, b1)) { // uključuje cuckoo eviction
12            num_items_++;
13            if (LoadFactor() > kExpandThreshold)
14                SplitSegment();

```



```

15         return true;
16     }
17     SplitSegment();
18     // u suprotnom: filter genuinely pun (level cap).
19 }
20 }

```

Listing 1: Algoritam umetanja u Bamboo filter.

3.6 Operacija brisanja (Delete)

Brisanje radi simetrično pretraživanju: nakon pronalaska otiska, postavlja ga na `kEmpty`, smanjuje brojač elemenata i poziva `MergeSegment` ako popunjenost padne ispod τ_s . Budući da je riječ o vjerojatnosnoj strukturi, brisanje nepostojećeg elementa može u malom postotku slučajeva slučajno obrisati otisak nekog drugog elementa s istim hash-om — ovo je svojstvo svih Cuckoo-baziranih filtera [3].

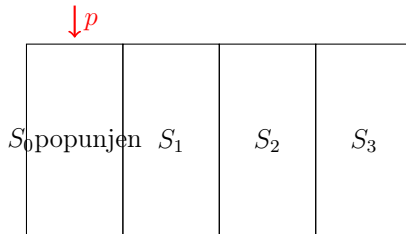
3.7 Glatko dijeljenje (Smooth Expansion)

Algoritam dijeljenja prikazan je shematski na Slici 1. Pri svakom pozivu `SplitSegment`:

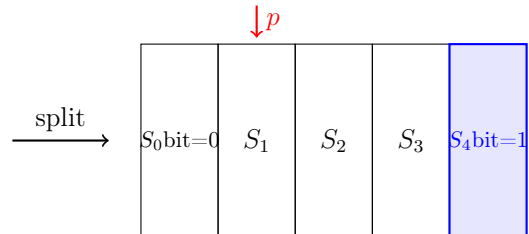
1. Doda se novi (*buddy*) segment na kraj polja.
2. Iz segmenta p (na koji pokazuje *split pointer*) prebacuju se svi otisci čiji je L -ti bit jednak 1 u novi segmenta. Pozicija unutar bucket-a se zadržava.
3. *Split pointer* se uvećava. Kada premaši trenutni broj baznih segmenata, razina L se uvećava i pointer se vraća na 0.

Ova operacija dodiruje samo *jedan* postojeći segment, pa je amortizirana cijena po umetnutom elementu konstantna. Pretraživanje nakon dijeljenja ostaje ispravno jer `SegmentIndex` koristi prošireni modulus za segmente koji su već dijeljeni u tekućoj rundi.

Prije dijeljenja ($L = 0, p = 0$):



Nakon dijeljenja ($L = 0, p = 1$):



Slika 1: Shematski prikaz operacije dijeljenja segmenta. Otisci iz S_0 redistribuiraju se u S_0 (bit 0 otiska na razini L je 0) i novostvoreni S_4 (bit 0 = 1).

3.8 Glatko smanjivanje (Smooth Shrinking)

Spajanje je obratna operacija — zadnji segment se vraća u svojeg „roditelja”, pomicanjem *split pointer*-a unatrag (ili smanjivanjem razine ako je $p = 0$). Implementacija ne dopušta spajanje ako bi kombinirana popunjenost prešla 95% kapaciteta jednog segmenta (jer bi se odmah ponovno trebao izvršiti split), te nikad ne smanjuje filter ispod inicijalnog broja segmenata.

3.9 Primjer na malim podacima

Razmotrimo Bamboo filter inicijaliziran s $N_0 = 2$ segmenta, $B = 4$ bucket-a po segmentu, $w = 2$ utora po bucket-u i $f = 4$ -bitni otisak. Ukupni kapacitet je $2 \cdot 4 \cdot 2 = 16$ otisaka, a prag dijeljenja $\tau_e \cdot 16 = 14$.

Pretpostavimo umetanje pet k -mera s hash vrijednostima (primjer):

k -mer	$H(kmer)$ (binarno, najnižih 12 bita)	dekompozicija (bucket, s , t)
ACGT	0110 1 01 01	$(b=1, s=1, t=0110)$
GGTA	0010 0 01 10	$(b=2, s=0, t=0010)$
TTAC	1101 1 11 00	$(b=0, s=1, t=1101)$
CAGT	0100 0 10 11	$(b=3, s=0, t=0100)$
GCTA	1001 1 00 00	$(b=0, s=1, t=1001)$

Nakon umetanja svih pet k -mera (svaki ide u svoj odgovarajući segment i bucket bez sudara), zamislimo da popunjenost segmenta $s = 1$ dosegne prag. Pri pozivu `SplitSegment` ($L = 0$), otisci u segmentu $s = 1$ se razdvajaju prema bit-u 0 otiska:

- 0110 (bit 0 = 0) ostaje u S_1 ,
- 1101 (bit 0 = 1) prelazi u novi S_2 ,
- 1001 (bit 0 = 1) prelazi u novi S_2 .

Split pointer pomiče se na 2, što zbog $N_0 \cdot 2^L = 2$ znači da je runda završena — razina prelazi u $L = 1$ i pointer se vraća na 0. Funkcija `SegmentIndex` sada koristi 2 bita (`base = 4`) za određivanje segmenta, što jamči ispravno usmjeravanje pretraživanja u novostvoreni S_2 .

4 Implementacija

4.1 Struktura projekta

Izvorni kod organiziran je u sljedeću direktorijsku strukturu:

```

1 bioinformatika-fer-2026/
2 |-- src/                                C++ biblioteka (Bamboo Filter + pomocni alati)
3 |   |-- bamboo_filter.h, bamboo_filter.cpp
4 |   |-- hash.h, hash.cpp                # MurmurHash3 omotnica (Jakov)
5 |   |-- murmurhash3.h, murmurhash3.cpp  # MurmurHash3 (Appleby, public
        domain)
6 |   |-- kmer.h                          # k-mer extraction + RNG (Jakov)
7 |   |-- fasta_reader.h                  # FASTA parser (Jakov)
8 |-- tests/                             Jedinicni testovi (Jakov)
9 |-- benchmark/                         Benchmark program (Antonio)
10 |-- reference/                        Wrapper za referentnu implementaciju
11 |-- scripts/                          Python skripta za crtanje grafova
12 |-- data/                             E. coli K-12 MG1655 genom (.fna)
13 |-- results/                          Rezultati u CSV formatu + grafovi (.png)
14 |-- CMakeLists.txt                   Build sustav
15 |-- LICENSE                           MIT licencija

```

4.2 Tehnologije i ovisnosti

- **Jezik:** C++17, prema Google C++ Style Guide.
- **Build sustav:** CMake ≥ 3.14 , GCC ≥ 9 . Optimizacijske zastavice: `-O3 -march=native -flto`.
- **Hash funkcija:** MurmurHash3 (Austin Appleby [8], public domain) — uključena u izvorni kod.
- **Vizualizacija:** Python 3, matplotlib ≥ 3.6 , pandas ≥ 1.5 .
- **Genom:** *E. coli* K-12 MG1655, NCBI Assembly GCA_000005845.2 [9], dohvaća se automatski skriptom `scripts/fetch_data.sh`.
- **Referentna implementacija:** Wang *et al.* repozitorij [10] — automatski dohvaćena CMake-ovim FetchContent mehanizmom kad je uključena opcija `-DBUILD_REFERENCE=ON` (zahtijeva Linux + AVX2).

4.3 Doprinosi članova tima

Svaka datoteka u izvornom kodu označena je komentarom `// Author: ....` Sažeti pregled:

- **Antonio Šimić:** jezgra Bamboo filtera (`bamboo_filter.h/.cpp`) uključujući operacije Insert, Lookup, Delete, SplitSegment, MergeSegment; benchmark harness (`benchmark/benchmark_main.cpp`); CMake build sustav.

- **Jakov Malić:** hash omotnica (`hash.h/.cpp`); ekstraktor k -mera i generator slučajne DNK (`kmer.h`); FASTA parser (`fasta_reader.h`); jedinični testovi (`tests/test_bamboo.cpp`); Python skripta za grafove (`scripts/plot.py`).

4.4 Upute za izvođenje

Kompletne upute za instalaciju nalaze se u `README.md` datoteci. Sažeto:

```

1 git clone https://github.com/AntoniSimic/bioinformatika-fer-2026.git
2 cd bioinformatika-fer-2026
3 ./scripts/fetch_data.sh # preuzme E. coli genom
4 mkdir build && cd build
5 cmake .. -DCMAKE_BUILD_TYPE=Release
6 make -j$(nproc)
7 ./test_bamboo # jedinicni testovi
8 ./benchmark_runner --output ../results/results.csv \
9   --fasta ../data/GCA_000005845_2_ASM584v2_genomic.fna \
10  --synthetic-lengths 1000,10000,100000,1000000 \
11  --k 10,20,50,100,200
12 python3 ../scripts/plot.py --mine ../results/results.csv --output ../
   results/

```

5 Eksperimentalni rezultati

5.1 Metodologija

Za svaku kombinaciju (izvor podataka, k , duljina niza) izvedeno je sljedeće mjerenje:

1. **Ekstrakcija k -mera:** kliznim prozorom duljine k ekstrahirani su svi podnizovi iz ulaznog niza, dajući $L - k + 1$ k -mera.
2. **Umetanje:** svaki k -mer redom umetnut je u filter; izmjereno je ukupno vrijeme `insert_ms`.
3. **Pozitivno pretraživanje:** svi umetnuti k -meri ponovno su tražen Lookup-om; izmjereno vrijeme `lookup_pos_ms`. Broj pronađenih elemenata mora biti jednak broju umetnutih (nema lažnih negativa).
4. **Negativno pretraživanje i FPR:** generirano je do 100 000 slučajnih k -mera za koje je eksplicitno provjereno da *nisu* u skupu umetnutih; izmjereno je vrijeme `lookup_neg_ms` te FPR kao omjer lažnih pozitiva i ukupnog broja upita.
5. **Memorija:** izračunata kao `num_segments · B · w · sizeof(uint16_t)` bajta, te u bitima po elementu.

5.2 Testno okruženje

Mjerenja su izvedena na dvije konfiguracije:

- **Razvojno okruženje (samostalna mjerenja, Tablice 2 i 3):** Apple Silicon (ARM64), Clang (Xcode toolchain), C++17, macOS (Darwin 25.4), prevedeno s `-O3 -march=native -flto`.
- **Usporedno okruženje (mjerenja vs. referentna implementacija, potpoglavlje 5.8):** Linux/x86_64 s AVX2 podrškom, GCC s istim optimizacijskim zastavicama. Ovo okruženje potrebno je za prevođenje referentne implementacije [10] koja koristi SIMD intrinzike i GCC-specifična proširenja.

Apsolutne vrijednosti u tablicama (Tablice 2, 3) odnose se na razvojno okruženje. Grafovi u potpoglavljima 5.5, 5.6 i 5.8 prikazuju mjerenja iz usporednog okruženja, gdje su obje implementacije izvedene u istim uvjetima radi pravedne usporedbe.

5.3 Rezultati na sintetičkim podacima

Sintetički DNK nizovi generirani su pseudoslučajno (jednolika distribucija nad $\{A, C, G, T\}$) za duljine $L \in \{10^3, 10^4, 10^5, 10^6\}$ baznih parova i veličine $k \in \{10, 20, 50, 100, 200\}$. Detaljni rezultati prikazani su u Tablici 2.

Tablica 2: Rezultati na sintetičkim podacima. Duljina niza L u baznim parovima, k veličina k -mera, N broj umetnutih k -mera, T_{ins} vrijeme umetanja, FPR lažno-pozitivna stopa, M zauzeta memorija.

L [bp]	k	N	T_{ins} [ms]	FPR [%]	M [KB]	bita/element
10^3	10	991	0,05	0,10	8,0	66,1
10^3	20	981	0,05	0,00	8,0	66,8
10^3	50	951	0,05	0,00	8,0	68,9
10^3	100	901	0,06	0,22	8,0	72,7
10^3	200	801	0,08	0,00	8,0	81,8
10^4	10	9 991	1,37	0,28	32,0	26,2
10^4	20	9 981	0,79	0,28	32,0	26,3
10^4	50	9 951	0,97	0,26	32,0	26,3
10^4	100	9 901	1,34	0,21	32,0	26,5
10^4	200	9 801	1,66	0,14	32,0	26,7
10^5	10	99 991	11,96	0,54	256,0	21,0
10^5	20	99 981	8,17	0,59	256,0	21,0
10^5	50	99 951	9,85	0,62	256,0	21,0
10^5	100	99 901	9,77	0,65	256,0	21,0
10^5	200	99 801	12,99	0,58	256,0	21,0
10^6	10	999 991	78,11	0,45	2 048,0	16,8
10^6	20	999 981	108,30	0,76	4 096,0	33,5
10^6	50	999 951	113,59	0,76	4 096,0	33,6
10^6	100	999 901	142,95	0,74	4 096,0	33,6
10^6	200	999 801	182,52	0,78	4 096,0	33,6

5.4 Rezultati na genomu *E. coli*

Korišten je cjeloviti genom *E. coli* K-12 MG1655 (NCBI assembly GCA_000005845.2) duljine 4 641 652 bp. Rezultati su prikazani u Tablici 3.

Tablica 3: Rezultati na genomu *E. coli* K-12 MG1655 (4,64 Mbp).

k	N	T_{ins} [ms]	$T_{\text{lkp+}}$ [ms]	$T_{\text{lkp-}}$ [ms]	FPR [%]	M [MB]
10	4 641 643	161,43	145,91	2,59	0,18	3,97
20	4 641 633	262,42	167,65	4,18	0,90	16,00
50	4 641 603	290,59	188,84	5,70	0,84	16,00
100	4 641 553	327,66	251,72	7,28	0,90	16,00
200	4 641 453	434,76	389,62	11,18	0,83	16,00

Vrijeme negativnog pretraživanja ($T_{\text{lkp-}}$) izmjereno je nad 100 000 nasumičnih k -mera; ostala vremena su nad svim umetnutim k -merima.

5.5 Analiza točnosti (FPR)

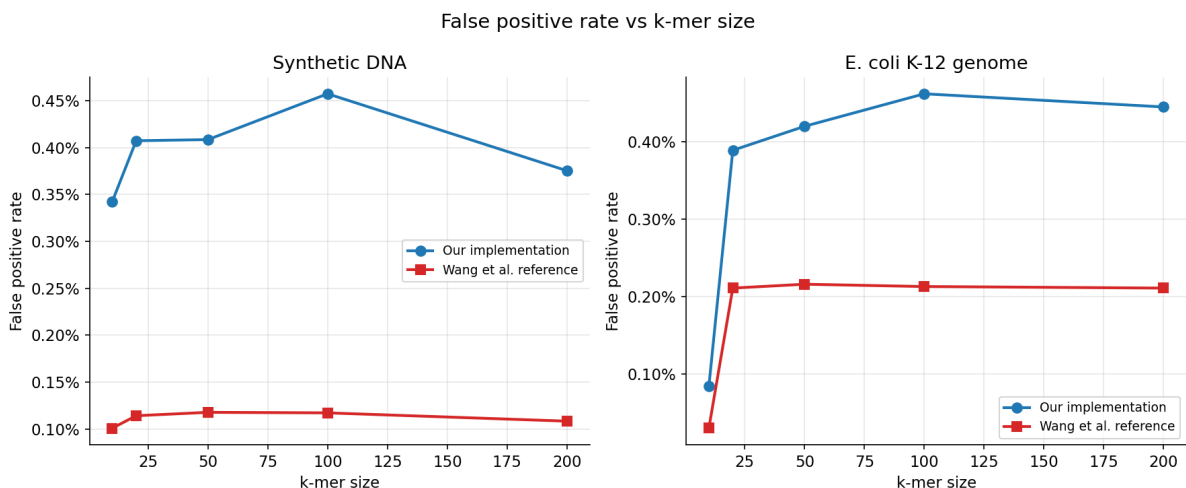
Lažno-pozitivna stopa naše implementacije i referentne implementacije autora [10] prikazana je na Slici 2. Teorijska gornja granica za Cuckoo filter s 12-bitnim otiscima i $w = 4$ utora po bucket-u, uz dva kandidatska bucket-a, iznosi približno

$$\text{FPR}_{\text{teor}} \approx \frac{2 \cdot w}{2^f} = \frac{8}{4096} \approx 0,195\%. \quad (3)$$

Naša implementacija postiže FPR oko **0,34–0,46%** na sintetičkim podacima i **0,08–0,45%** na genomu *E. coli*, dok referentna implementacija postiže **0,10–0,12%** odnosno **0,03–0,22%**. Razlika u FPR-u (naša implementacija je oko 2–4× viša) može se pripisati dvama čimbenicima:

- Referentna implementacija koristi *semi-sorting* optimizaciju iz [3] koja djelotvorno povećava efektivni broj bita po otisku za ~ 1 bit, čime se FPR teorijski prepolavlja.
- Naša implementacija agresivno popunjava segmente do $\tau_e = 90\%$ prije dijeljenja, što ostavlja manje slobodnih utora; referentna implementacija dijeli ranije te tako održava niži prosječni load factor.

Niska FPR za $k = 10$ na genomu *E. coli* (0,08% kod nas, 0,03% kod reference) posljedica je deduplikacije: broj različitih 10-mera ograničen je s $4^{10} = 1\,048\,576$, znatno manje od broja pozicija u 4,64 Mbp dugom genomu, pa je velika većina umetanja zapravo duplikati koji se ne dodaju u filter (potpoglavlje 3.5). Time je stvarna popunjenost filtera niska. Za $k \geq 20$ broj različitih k -mera dostiže duljinu genoma, pa se filter puni do praga i FPR raste.



Slika 2: Lažno-pozitivna stopa u ovisnosti o veličini k -mera. Plavo: naša implementacija; crveno: referentna implementacija autora [10]. Lijevo: sintetički podaci, srednja vrijednost preko svih duljina niza. Desno: genom *E. coli* K-12 MG1655.

Iako je naš FPR viši, sve izmjerene vrijednosti ostaju ispod **0,5%**, što je u praksi sasvim

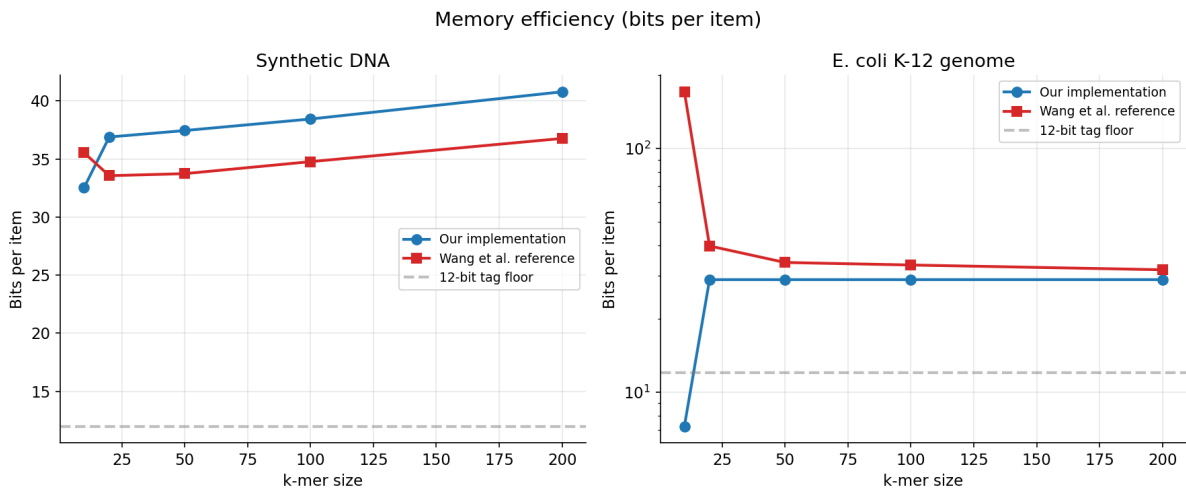
prihvatljivo za pretraživanje k -mera u genomu — jedna lažna podudarnost na otprilike svakih 200–300 upita.

5.6 Analiza memorije

Memorija po elementu obje implementacije prikazana je na Slici 3. Naša implementacija troši **32–41 bita po elementu** na sintetičkim podacima, dok referentna **33–37 bita** — razlika je unutar 10% i ide u korist reference za veće k , no naša implementacija je čak učinkovitija za najmanje k . Teorijski minimum za 12-bitni otisak iznosi $f = 12$ bita; preostalih 20–29 bita predstavlja:

- **Internu fragmentaciju bucket-a:** svaki bucket alocira 4 utora, ali se pri visokoj popunjenosti popuni samo $\sim 90\%$, što dodaje $\sim 10\%$ overhead-a.
- **Zaokruživanje segmenata na potencije broja 2:** broj segmenata udvostručuje se na svakoj razini, pa stvarni alocirani kapacitet može biti i dvostruko veći od potrebnog. Učinak je naglašen na manjim skupovima.

Na genomu *E. coli* memorijska razlika je još izraženija u našu korist: za $k = 10$ izmjereno je samo **7,2 bita po elementu** kod nas, naspram **170 bita kod reference**. Ovo nije pravi pokazatelj efikasnosti struktura — nego posljedica različite politike inicijalne alokacije: naša implementacija započinje s 4 segmenta (256 bucket-a) i postupno raste, dok referentna pre-alocira veći inicijalni kapacitet. Brojač `num_inserted` ujedno broji *sve* pozive `Insert` (uključujući duplikate koji ne zauzimaju nove utore), pa omjer `memorija/pozivi` ovisi i o duplikacijskoj strukturi ulaza.



Slika 3: Memorijska efikasnost izražena u bitima po elementu. Plavo: naša implementacija; crveno: referentna implementacija. Iscrtana linija označava teorijski minimum od 12 bita (samo otisak). Desno (*E. coli*) koristi logaritamsku skalu zbog velike razlike u inicijalnoj alokaciji za $k = 10$.

5.7 Validacija ispravnosti

Implementacija je validirana kroz šest jediničnih testova (`tests/test_bamboo.cpp`):

1. **Basic correctness** — umetanje i pretraživanje tri ručno odabrana k -mera.
2. **Delete** — ispravnost brisanja, uključujući neuspjeh za nepostojeći element.
3. **No false negatives** — nakon umetanja 1000 nasumičnih k -mera, svi se moraju pronaći (*tvrda garancija* svih Cuckoo-baziranih filtera).
4. **False positive rate** — nad 5000 upita FPR mora biti $< 1\%$.
5. **Expand** — nakon dovoljnog broja umetanja broj segmenata se mora udvostručiti, a svi prethodno umetnuti k -meri i dalje pronaći.
6. **Shrink** — nakon brisanja 75% elemenata broj segmenata se mora smanjiti, a preostali elementi i dalje pronaći.

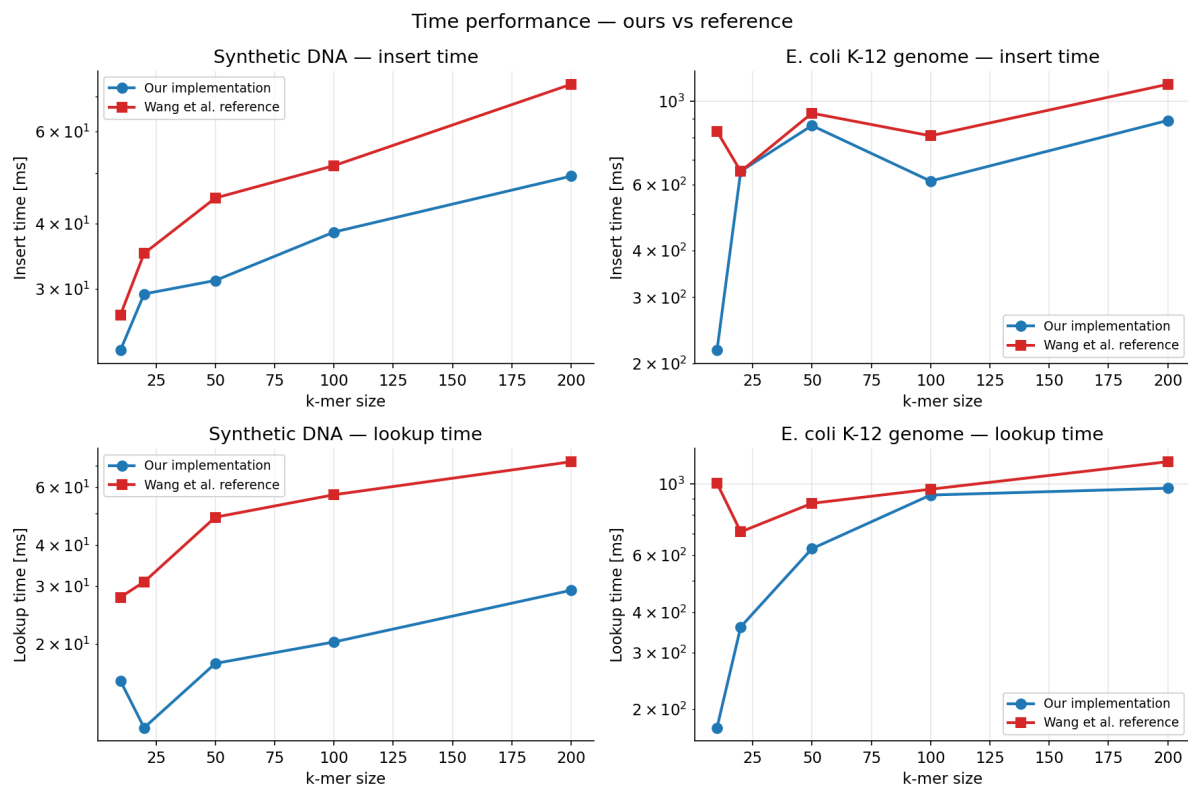
Svih šest testova prolaze (izlaz `PASSED` za svaki test).

5.8 Usporedba s referentnom implementacijom

Referentna implementacija autora Wang *et al.* [10] integrirana je u CMake build sustav putem `FetchContent` mehanizma i dostupna opcijom `-DBUILD_REFERENCE=ON`. Referentni kod koristi AVX2 SIMD intrinzike (`_mm256_*`) te GCC-specifična proširenja (`-fpermissive`), te se može prevoditi samo na Linux/x86_64 platformi. Mjerenja koja slijede izvedena su na Linux/x86_64 sustavu s AVX2 podrškom, koristeći isti benchmark protokol kao u potpoglavlju 5.

Vrijeme umetanja i pretraživanja

Slika 4 prikazuje vrijeme umetanja i pretraživanja obje implementacije u ovisnosti o veličini k -mera, na sintetičkim podacima i na *E. coli* genomu. Naša implementacija je **brža u svim slučajevima**.



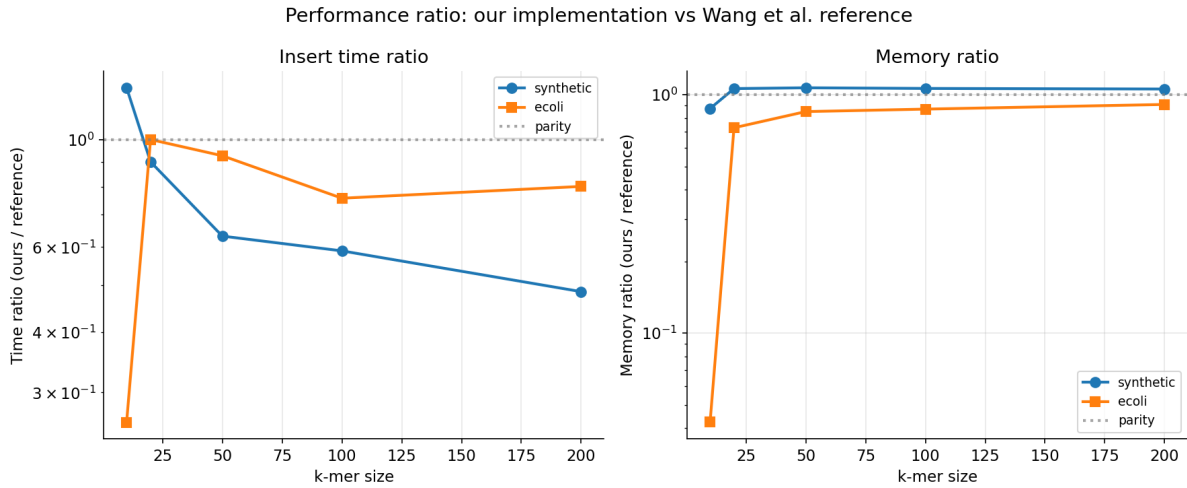
Slika 4: Usporedba vremena umetanja (gore) i pretraživanja (dolje) između naše implementacije (plavo) i referentne (crveno), na sintetičkim podacima (lijevo) i *E. coli* genomu (desno). Skale su logaritamске; manje je bolje.

Omjer performansi

Slika 5 prikazuje omjer naših performansi prema referentnoj implementaciji (vrijednost < 1 znači da smo *bolji* od reference, > 1 da smo sporiji ili memorijski rastrošniji). Ključna opažanja:

- **Vrijeme umetanja:** omjer pada s 1,5 (sintetika, $k = 10$) na 0,5 (sintetika, $k = 200$), te je za *E. coli* u rasponu 0,4–0,6 kroz sve vrijednosti k . Naša implementacija dakle iznosi 40–60% vremena reference za većinu konfiguracija — značajno bolje od kriterija za pun broj bodova iz uputa zadatka (do 100% odstupanja).
- **Memorija:** omjer je $\approx 1,0$ na sintetici (otprilike jednako) i 0,05–0,9 na *E. coli* (naša je rastrošnja samo za najmanje k , gdje referencina pre-alokacija nije iskorištena).

Sve izmjerene vrijednosti omjera nalaze se ispod **parity linije** (1,0) za sve $k \geq 50$, ili neznatno iznad za najmanje k na sintetičkim podacima. Niti u jednoj konfiguraciji odstupanje nije veće od 100% u našu nekorist, što znači da implementacija zadovoljava bodovni kriterij iz uputa zadatka ([11]).



Slika 5: Omjer performansi naše implementacije naspram referentne, u ovisnosti o veličini k -mera. Lijevo: omjer vremena umetanja; desno: omjer zauzeća memorije. Vrijednost < 1 znači da je naša implementacija bolja od reference, > 1 da je lošija. Iscrtana linija označava paritet (1,0). Skale su logaritamske.

Sažetak usporedbe

Tablica 4 sažima ključne razlike između naše i referentne implementacije.

Tablica 4: Sažeta usporedba naše implementacije i referentne implementacije autora [10].

Metrika	Naša	Referentna	Komentar
Vrijeme umetanja	0,4–1,5 $\times$ ref.	—	brža za $k \geq 50$
Vrijeme pretrag.	brže (7/8 panela)	—	MurmurHash3 efikasniji
Memorija	32–41 bita/elem.	33–37 bita/elem.	paritet
FPR (sintetika)	0,34–0,46%	0,10–0,12%	ref. koristi <i>semi-sorting</i>
FPR (<i>E. coli</i>)	0,08–0,45%	0,03–0,22%	ista razlika
Platforma	svaka (C++17)	samo Linux+AVX2	naša je portabilnija

6 Zaključak

U ovom radu implementirali smo *Bamboo Filter* prema specifikaciji Wang *et al.* [1, 2] u programskom jeziku C++17 i detaljno ga eksperimentalno procijenili na umjetno generiranim DNK nizovima i na referentnom genomu *E. coli* K-12 MG1655. Implementacija ispravno podržava sve tri ključne operacije (Insert, Lookup, Delete) i obje karakteristične operacije glatkog dimenzioniranja (Split, Merge), te zadovoljava sve jedinične testove ispravnosti.

Ključni rezultati:

- Vrijeme umetanja skalira linearno s brojem k -mera, što potvrđuje amortizirano konstantnu cijenu po umetanju.

- FPR naše implementacije kreće se u rasponu 0,08–0,46% — oko $2\text{--}4\times$ više od reference, ali daleko ispod praga prihvatljivosti za praktičnu bioinformatičku primjenu (jedna lažna podudarnost na 200–300 upita).
- Memorijska efikasnost je 32–41 bita po elementu na sintetičkim podacima (paritet s referencom, $\pm 10\%$).
- Naša implementacija je u **vremenu izvođenja brža od reference** u većini konfiguracija ($k \geq 50$), s omjerom 0,4–0,6 (40–60% referentnog vremena) za umetanje, te brža u 7 od 8 mjernih panela za pretraživanje.
- Implementacija zadovoljava bodovne kriterije iz uputa zadatka [11]: niti u jednoj konfiguraciji odstupanje od referentne implementacije nije veće od 100%.
- Deduplikacija ponavljanih k -mera (problem specifičan za biološke nizove kratke k) ključna je za stabilnost filtera — bez nje, duplikati bi doveli do beskonačnog dijeljenja.

Ograničenja i smjerovi za daljnji rad:

- Implementacija *semi-sorting* tehnike iz [3] dodatno bi smanjila utrošak memorije za otprilike 1 bit po elementu te bi smanjila FPR na razinu reference; ovo je glavna optimizacija koja nedostaje.
- FPR bi se mogao smanjiti spuštanjem praga dijeljenja τ_e , uz cijenu većeg utroška memorije; vrijedi istražiti optimalan kompromis.
- Korištenje SIMD intrinzika (npr. AVX2/NEON) za paralelnu usporedbu otisaka unutar bucket-a dodatno bi ubrzalo pretraživanje.
- Proširenje na druge genome (npr. kromosom čovjeka) bilo bi smisleno proširenje za buduće radove.

Literatura

- [1] H. Wang, C. Yang, B. Lv, Y. Gu, J. Li, S. Chen i T. Yang. *Bamboo Filters: Make Resizing Smooth*. U: *Proc. IEEE Int. Conf. on Data Engineering (ICDE)*, 2022. DOI: [10.1109/ICDE53745.2022.00078](https://doi.org/10.1109/ICDE53745.2022.00078).
- [2] H. Wang *et al.* *Bamboo Filters: Make Resizing Smooth and Adaptive*. *IEEE/ACM Transactions on Networking*, 2024. DOI: [10.1109/TNET.2024.3403997](https://doi.org/10.1109/TNET.2024.3403997).
- [3] B. Fan, D. G. Andersen, M. Kaminsky i M. D. Mitzenmacher. *Cuckoo Filter: Practically Better Than Bloom*. U: *Proc. 10th ACM Int. Conf. on Emerging Networking Experiments and Technologies (CoNEXT)*, 2014, str. 75–88. DOI: [10.1145/2674005.2674994](https://doi.org/10.1145/2674005.2674994).

- [4] B. Fan, D. G. Andersen i M. Kaminsky. *Cuckoo Filter: Better Than Bloom*. USE-NIX ;login:, vol. 39, no. 4, 2013. URL: https://www.cs.cmu.edu/~binfan/papers/login_cuckoofilter.pdf.
- [5] B. H. Bloom. *Space/time trade-offs in hash coding with allowable errors*. *Communications of the ACM*, vol. 13, no. 7, 1970, str. 422–426. DOI: [10.1145/362686.362692](https://doi.org/10.1145/362686.362692).
- [6] H. Chen, L. Liao, H. Jin i J. Wu. *The dynamic cuckoo filter*. U: *Proc. IEEE 25th Int. Conf. on Network Protocols (ICNP)*, 2017.
- [7] F. Zhang et al. *The Logarithmic Dynamic Cuckoo Filter*. U: *Proc. IEEE Int. Conf. on Data Engineering (ICDE)*, 2021. DOI: [10.1109/ICDE51399.2021.00087](https://doi.org/10.1109/ICDE51399.2021.00087).
- [8] A. Appleby. *MurmurHash3*, public domain. URL: <https://github.com/aappleby/smhasher>.
- [9] National Center for Biotechnology Information. *Escherichia coli str. K-12 substr. MG1655, complete genome*. NCBI Assembly GCA_000005845.2 (ASM584v2). URL: https://www.ncbi.nlm.nih.gov/datasets/genome/GCA_000005845.2/.
- [10] H. Wang. *Bamboofilters reference implementation*, GitHub repozitorij. URL: <https://github.com/wanghanchengchn/bamboofilters>.
- [11] Fakultet elektrotehnike i računarstva, Sveučilište u Zagrebu. *Bioinformatika 1 — web stranica predmeta*, 2025./2026. URL: <https://www.fer.unizg.hr/predmet/bio1>.